
UNIT 11: AI PROGRAMMING WITH PYTHON

Structure	Page No.
11.1 Introduction to Python for AI	322
11.2 Objectives	323
11.3 Python Libraries for AI	323
11.4 Hands-on Implementations of:	349
11.4.1 N-Queen Problem	349
11.4.2 Min-Max Algorithm and Alpha-Beta Pruning	351
11.4.3 Breadth First Search	357
11.4.4 Depth First Search	359
11.4.5 Best First Search	360
11.4.6 A* Algorithm	361
11.4.7 AO* Algorithm	364
11.4.8 Iterative Deepening A*	366
11.5 Model Evaluation and Hyperparameter Tuning in Python	368
11.6 Summary	370
11.7 Solutions/Answers	370
11.8 Further Readings	371

11.1 INTRODUCTION TO PYTHON FOR AI

Python is a high-level, beginner-friendly, general-purpose programming language known for its readable syntax and wide range of applications. It has emerged as an important programming language for Artificial Intelligence (AI) and Machine Learning (ML).

Python has been adopted by most of the researchers, developers, and organizations to build AI-based applications because of the following reasons:

1. Python has easy syntax and human-readable code, which makes it accessible for both beginners and experienced programmers. AI development often involves complex algorithms and mathematical computations, and Python's ease of writing and debugging reduces development time and effort.
2. Python has a collection of several libraries that simplify AI and ML development. Some of the most widely used libraries include NLTK, OpenCV, Pandas, etc.
3. Python has a massive global community that actively contributes to open-source projects, which ensures continuous improvements and support. The availability of the extensive documentation, forums, and online courses makes learning and troubleshooting easy.
4. Python is a platform-independent language, which means that the applications developed in Python can run on various operating systems without any modification. This makes it an excellent choice for deploying AI solutions across different environments.

AI has multiple approaches to make the machines intelligent. Earlier, AI algorithms were based on rule-based systems and search techniques which were using logical reasoning and problem-solving methods. Later, the field has

shifted towards machine learning, where computers can learn the patterns from the data instead of relying solely on pre-defined rules.

Since Python has a rich set of libraries, it supports all these approaches. From building expert systems using rules to search-based problem-solving, and to modern deep learning models, Python provides a unified platform that can connect the history and future of AI. Some of the applications of Python in AI are:

- *Pathfinding in Robotics and Games* - Robots use state space search algorithms: A* or BFS/DFS to navigate in an environment.
- *Puzzle Solving* - Classic AI problems like the 8-puzzle, Sudoku, or N-Queens are solved using state-space search in Python.
- *Planning in Education Systems* - Intelligent tutoring systems search the space of teaching actions to lead students from current knowledge to mastery.
- *Machine Learning and Deep Learning* - Python enables the development of intelligent systems that can learn from data, recognize patterns, and make predictions. It is used in analytics, recommendation systems, and fraud detection.
- *Natural Language Processing (NLP)* - With libraries like NLTK, spaCy and transformers, Python is extensively used in NLP tasks such as sentiment analysis, chatbot development and automated translation.
- *Computer Vision* - Python's OpenCV library and deep learning frameworks help in image processing, facial recognition and object detection, making it essential in security, healthcare and autonomous driving applications.
- *Robotics and Automation* - Python plays a crucial role in robotic process automation (RPA) and autonomous systems, allowing machines to perform tasks with minimal human intervention.
- *Reinforcement Learning* - Python is used in AI-driven gaming, robotics and decision-making systems through reinforcement learning algorithms.

11.2 OBJECTIVES

After going through this unit, you should be able to:

- Understand how to use Python for AI-based applications
- Use available Python libraries
- Implement AI-based algorithms in Python

11.3 PYTHON LIBRARIES FOR AI

Python has a rich set of libraries to simplify various AI-based complex tasks. Python library such as Numpy and Pandas is used to process large amount of numerical data. Matplotlib is the library used for visualization of results. In rule-based AI systems, Python offers libraries such as Pyke and Experta, which help developers create knowledge bases and inference engines for decision-making. These libraries make it easy to represent rules, perform logical reasoning, and

build expert systems without starting from scratch. For state-space search, Python provides efficient implementations of graph and search algorithms through libraries like NetworkX, enabling exploration of paths, optimization problems, and heuristic-based approaches such as A*, IDA*, or even game-tree search using minimax.

Further, scikit-learn is one of the most widely used libraries for machine learning (ML). It offers a number of algorithms for classification, regression, clustering, dimensionality reduction and reinforcement learning. For deep learning (DL), Python provides support for the frameworks like TensorFlow and PyTorch which can be used to build neural networks, reinforcement learning systems and large-scale AI applications. Natural Language Processing (NLP) is also supported by libraries such as NLTK, spaCy and transformers (Hugging Face), which provide tools for sentiment analysis, chatbot development and machine translation. In the field of computer vision, OpenCV along with deep learning frameworks can be used for the tasks such as image recognition, medical imaging analysis and autonomous vehicle perception. Additionally, Python supports reinforcement learning using libraries like Stable Baselines3 and Ray RLLib which simplifies building agents that learn through trial and error in simulated environments. Following section will discuss some of these libraries in detail.

11.3.1 NumPy

AI models work with large amounts of numerical data and handling this efficiently is a key challenge. In Python, NumPy (short for Numerical Python) is a library that provides tools to work with arrays, matrices and numerical operations. It is the foundation of many other scientific and machine learning libraries like Pandas, SciPy, TensorFlow and PyTorch. It provides efficient data structures, mathematical functions, random number generation and linear algebra tools. It can be used for preprocessing data, performing matrix operations, or implementing simple AI models.

If you are using Anaconda, NumPy is already installed. Otherwise, you can install it using:

```
pip install numpy
```

To import NumPy library:

```
import numpy as np
```

Here, np is just an alias to make typing shorter. You can assign any name to create the alias.

For AI, we often work with **2D arrays** (matrices) and **3D arrays** (image data, sequences, etc.). This can be handled using NumPy arrays. This is similar to Python lists but is more powerful. You can create NumPy arrays in several ways:

```
#CODE: Arrays in python
import numpy as np

# From a Python list
arr = np.array([1, 2, 3, 4])
print("Python list is: ",arr)

# Multi-dimensional array
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2-D array is ",arr2d)

# Using built-in functions
zeros = np.zeros((3,3))      # 3x3 matrix of zeros
ones = np.ones((2,2))       # 2x2 matrix of ones
randoms = np.random.rand(3,3) # 3x3 matrix with
random numbers
print("3x3 matrix of zeros", zeros)
print("2x2 matrix of ones",ones)
print("3x3 matrix with random numbers", randomnesss)
```

OUTPUT:

```
Python list is:
[1 2 3 4]

2-D array is
[[1 2 3]
 [4 5 6]]

3x3 matrix of zeros
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

2x2 matrix of ones
[[1. 1.]
 [1. 1.]]

3x3 matrix with random numbers
[[0.0368857  0.65398728 0.80446459]
 [0.20939835 0.79966979 0.92998546]
 [0.69515611 0.52541098 0.21407825]]
```

Every array has some properties that are useful such as shape, dimensions, data types, size, etc. These properties can be seen as follows:

```
#CODE: Arrays properties in python

a2d = np.array([[1, 2, 3], [4, 5, 6]]) #initialize
a 2-D array
```

```

print("Shape (rows, columns): ", a2d.shape) # Shape
(rows, columns)
print("Number of dimensions: ", a2d.ndim) # Number
of dimensions
print("Data type: ", arr2d.dtype) # Data type
print("Total number of elements: ", a2d.size) #
Total number of elements

```

OUTPUT:

```

Shape (rows, columns): (2, 3)
Number of dimensions: 2
Data type: int64
Total number of elements: 6

```

Once array is created, next step is to access its element either one by one (using indexing) or a subset of elements together (using slicing).

```

#CODE: Arrays Indexing and Slicing

a = np.array([10, 20, 30, 40, 50])
print("First element:", a[0]) # First element
print("Last element:", a[-1]) # Last element

b = np.array([[1, 2, 3], [4, 5, 6]])
print("Row 0, Column 1:", b[0, 1]) # Row 0, Column
1 -> 2

c = np.array([10, 20, 30, 40, 50])
print("Elements from index 1 to 3:", c[1:4]) #
Elements from index 1 to 3
print("First three elements:", c[:3]) # First
three elements
print("From index 2 to end:", c[2:]) # From index
2 to end

d = np.array([[1,2,3], [4,5,6], [7,8,9]])
print("First two rows, last two columns:", d[0:2,
1:3]) # First two rows, last two columns

```

OUTPUT:

```

First element: 10

Last element: 50

Row 0, Column 1: 2

Elements from index 1 to 3: [20 30 40]

First three elements: [10 20 30]

From index 2 to end: [30 40 50]

```

```
First two rows, last two columns:  
[[2 3]  
 [5 6]]
```

Further, NumPy arrays allow vectorized operations, meaning we can perform arithmetic directly on arrays without loops. This ability is very important in AI, where datasets contain thousands of numbers and we need to process them quickly.

```
# CODE: Array Operations  
  
x = np.array([1, 2, 3])  
y = np.array([4, 5, 6])  
  
print("Addition of two Vectors:", x + y)  
print("Subtraction of two Vectors:", x - y)  
print("Multiplication of two Vectors:", x * y)  
print("Division of two Vectors:", x / y)  
  
print("Scalar multiplication of each element by 2:",  
x * 2)  
print("Taking square of each element:", x ** 2)
```

OUTPUT:

```
Addition of two Vectors: [5 7 9]  
Subtraction of two Vectors: [-3 -3 -3]  
Multiplication of two Vectors: [ 4 10 18]  
Division of two Vectors: [0.25 0.4  0.5 ]  
Scalar multiplication of each element by 2: [2 4 6]  
Taking square of each element: [1 4 9]
```

Some other useful functions in NumPy that are heavily used for data preprocessing and feature engineering in AI are demonstrated in the following code.

```
#CODE  
  
arr = np.array([1, 2, 3, 4, 5])  
print(np.sum(arr))          # 15  
print(np.mean(arr))         # 3.0  
print(np.std(arr))          # Standard deviation  
print(np.max(arr))          # 5  
print(np.min(arr))          # 1
```

```
#For multi-dimensional arrays:
mat = np.array([[1,2,3],[4,5,6]])
print(np.sum(mat, axis=0)) # Column-wise sum    [5 7
9]
print(np.sum(mat, axis=1)) # Row-wise sum      [6 15]
```

NumPy also provides functions for different matrix operations as follows:

```
#CODE: Matrix Operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = np.dot(A, B)
print("Matrix multiplication:", C)

# Transpose
print("Transpose of A:", A.T)

# Determinant
print("Determinant of A:", np.linalg.det(A))

# Inverse
print("Inverse of A:", np.linalg.inv(A))
```

OUTPUT:

```
Matrix multiplication:
[[19 22]
 [43 50]]
Transpose of A:
[[1 3]
 [2 4]]

Determinant of A: -2.0000000000000004

Inverse of A:
[[-2.   1. ]
 [ 1.5 -0.5]]
```

AI models often require randomness for initializing weights, shuffling data, or splitting datasets. The random module of NumPy allows to generate the random data. The code for the same as follows:

```
#CODE: random module

# Random numbers between 0 and 1
print(np.random.rand(5))

# Random integers
```

```
print(np.random.randint(1, 10, size=5))

# Normal distribution (mean=0, std=1)
print(np.random.randn(5))
```

OUTPUT:

```
# Random numbers between 0 and 1
[0.89182018 0.173299 0.64496502 0.17889427
0.94136995]
# Random integers
[6 4 8 1 3]
# Normal distribution (mean=0, std=1)
[-0.87590679 -0.24440562 0.05706411 -0.77204154 -
0.96940411]
```

Broadcasting can be used to perform operations between arrays of different shapes. This concept can save time and memory and is commonly used in AI model computations.

```
#CODE: Broadcasting

a = np.array([[1,2,3],[4,5,6]])
b = np.array([1,2,3])

print(a + b) # array b is added to both rows of a
```

OUTPUT:

```
[[2 4 6]
 [5 7 9]]
```

AI models train better if data is normalized. Though, there is no direct function in Numpy for normalization, same can be done in one go by writing the formula.

```
#CODE: Data Normalization

data = np.array([10, 20, 30, 40, 50])
normalized = (data - np.mean(data)) / np.std(data)
print("Normalized data is ", normalized)
```

OUTPUT:

```
Normalized data is [-1.41421356 -0.70710678 0.
0.70710678 1.41421356]
```

Designing AI applications using NumPy has a number of advantages. It provides high speed and efficiency as its array operations are significantly faster than Python lists which makes it suitable for handling large datasets. NumPy is also memory-efficient which helps in reducing the storage requirement while processing massive amounts of data. With its range of built-in mathematical and

statistical functions, it simplifies the complex computations. Moreover, NumPy is compatible with other popular AI and data science libraries such as Pandas, SciPy, TensorFlow and PyTorch.

Despite of its advantages, it does have some limitations as well. It can work only on single CPU which restricts its scalability for very large datasets. It also lacks higher-level abstractions such as models, layers and training utilities that specialized AI frameworks like TensorFlow and PyTorch provide.

11.3.2 Pandas

AI is all about making machines learn from data and make decisions. But before staring the model training, we need to organize, clean and analyse the data. This step is called data preprocessing. It is one of the important step of any machine learning model so that the model can learn the pattern from clean data. Usually, this step takes lots of time in AI projects. To help the programmers, one of the most powerful tools for preprocessing in Python is the Pandas library.

Pandas is an open-source library which provides flexible data structures and data analysis tools. It can handle large datasets, clean the missing values, merge different files and perform the statistical analysis with just a few lines of code. It acts as a bridge between the raw data and the machine learning models.

If you are using Anaconda, Pandas is already installed. Otherwise, you can install it using:

```
pip install pandas
```

To import Pandas library:

```
import pandas as pd
```

Here, pd is just an alias to make typing shorter. You can assign any name to create the alias.

The core data structures supported by Pandas are:

1. Series – A one-dimensional labeled array.
2. DataFrame – A two-dimensional labeled data structure (like a table).

A **Series** is like a column in a table. It can be defined with default index starting from 0 or with custom indices as follows:

```
#CODE: Series
import pandas as pd

data_dafult_index = pd.Series([10, 20, 30, 40])
print("Series with default indices",
data_dafult_index)
```

```
data_custom_index = pd.Series([10, 20, 30],
index=['a', 'b', 'c'])
print("Series with custom
indices",data_custom_index)
```

OUTPUT:

```
Series with default indices
0    10
1    20
2    30
3    40
dtype: int64

Series with custom indices
a    10
b    20
c    30
dtype: int64
```

Dataframe in Pandas is like a table with multiple rows and columns. It helps in handling large dataset.

```
#CODE: Dataframes

data = {
    'Name': ['P', 'Q', 'R'],
    'Age': [18, 27, 20],
    'Score': [85, 90, 88]
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT:

	Name	Age	Score
0	P	18	85
1	Q	27	90
2	R	20	88

Pandas supports reading and writing data from CSV, xlsx and JSON files. The path with filename to be mentioned can be absolute path or the relative path depending upon the location of your current Python file. The code with relative path is as follows:

```
# Reading CSV file
df = pd.read_csv("data.csv")

# Writing to CSV file
df.to_csv("output.csv", index=False)
```

```
# Reading Excel file
df = pd.read_excel("data.xlsx")

# Reading JSON file
df = pd.read_json("data.json")
```

Once data is read and stored in dataframes, next step is to explore the dataset. Pandas provides a number of functions for the same. These functions help us quickly understand the dataset's structure.

```
#CODE: Data exploration

print(df.head())      # First 5 rows
print(df.tail())      # Last 5 rows
print(df.info())      # Information about dataset
print(df.shape)       # (rows, columns)
print(df.columns)     # List of column names
print(df.describe())  # Statistical summary such as
count, mean, min, max etc
```

Following code explains how to handle columns and rows in the dataframe.

```
#CODE: Data Selection

#Selecting Columns
print(df['Name'])      # Single column
print(df[['Name', 'Age']]) # Multiple columns

#Selecting Rows
print(df.iloc[0])     # First row by position
print(df.loc[1])      # Row with index label 1

#Conditional Selection
print(df[df['Age'] > 23]) # Filter rows where Age > 23
```

Real-world datasets are rarely clean. Pandas provides various functions to clean the data as follows:

```
#CODE: Data Cleaning

df.isnull().sum()      # Count missing values
df = df.dropna()       # Drop rows with missing values
df = df.fillna(0)      # Replace missing with 0
df = df.drop_duplicates() # Handling Duplicates
df['Age'] = df['Age'].astype(float) #Changing Data Types
```

AI models often require transformation of data. It can be done as follows:

```
#CODE: Data transformation
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'Score': [85, 90, 88]
}
df = pd.DataFrame(data)
print("Data before transformation", df);

# Renaming Columns
df.rename(columns={'Score':'Marks'}, inplace=True)
# Sorting Data
df.sort_values('Age', ascending=False, inplace=True)
# Applying Functions
df['Marks'] = df['Marks'].apply(lambda x: x/10)

print("Data after transformation", df)
```

OUTPUT:

```
Data before transformation
Name Age Score
0    Alice   24    85
1     Bob   27    90
2  Charlie   22    88

Data after transformation
Name Age Marks
1     Bob   27   9.0
0    Alice   24   8.5
2  Charlie   22   8.8
```

Here, `inplace=True` is used in `rename` and `sort_values` function. It means that changes will be reflected in the original dataframe. As it can be seen in the output, the data gets sorted in the dataframe itself. By default, this value is false, indicating that changes are temporary. Also, `ascending=False` means that it is sorted in descending order. Further, `apply()` method is used to apply lambda function. It takes one argument, `x` and returns the value of `x` divided by 10.

Pandas provides support to concatenate, merge or group the data as follows:

```
#CODE: Data Combining

#Concatenation
df1 = pd.DataFrame({'A':[1,2], 'B':[3,4]})
df2 = pd.DataFrame({'A':[1,6], 'B':[7,8]})
result1 = pd.concat([df1, df2])
print("Concatenated Dataframe is ")
```

```
print(result1)

#Merging
result2 = pd.merge(df1, df2, on='A')
print("Merged Dataframe is ")
print(result2)
```

OUTPUT:

```
Concatenated Dataframe is
   A  B
0  1  3
1  2  4
0  1  7
1  6  8
Merged Dataframe is
   A  B_x  B_y
0  1    3    7
```

Here, `pd.concat()` function concatenates two or more dataframes based on columns. Similarly, `pd.merge` function allows us to combine 2 dataframes based on a certain column, which must be present in both dataframes. So, in the above code, the data is combined based on column “A” by fetching values corresponding to “A” from both the dataframes.

AI depends on statistics. Pandas provides built-in methods:

```
#CODE: Data Analysis

print("Average Marks:", df['Marks'].mean()) # Average
print("Median:", df['Marks'].median()) # Median
print("Standard deviation:", df['Marks'].std()) # Standard deviation
print("Correlation", df['Marks'].corr(df['Age'])) # Correlation
```

OUTPUT:

```
Average Marks: 8.766666666666667
Median: 8.8
Standard deviation: 0.2516611478423584
Correlation 0.49999999999999984
```

Pandas provides several advantages in AI by offering an easy-to-use interface with simple syntax, making the data manipulation and analysis easier. It can efficiently handle large datasets and offers flexible data structures such as Series and DataFrames for organizing and processing the information. Pandas also supports multiple data formats such as CSV, Excel and JSON which makes it useful for real-world data handling. Moreover, its strong integration with NumPy, Scikit-learn and TensorFlow ensures smooth compatibility. However, Pandas also has some limitations. It mainly works on a single machine’s

memory, which makes it challenging to handle extremely large or distributed datasets.

11.3.3 Matplotlib

AI is not just about building the models. A major part of AI projects involves understanding and visualizing the data. Visualization helps us to see the patterns, trends and relationships among data that are difficult to identify in the raw numbers. It also helps to communicate and display the results effectively to others. In Python, one of the most powerful visualization libraries is Matplotlib. It is an open-source library that allows us to create a wide range of static, animated and interactive plots. Generally, Matplotlib is used for

1. Exploratory Data Analysis (EDA) – Understanding dataset distributions, outliers, correlations and trends.
2. Model Training Insights – Plotting loss curves and accuracy during training.
3. Feature Importance – Visualizing which features contribute most to predictions.
4. Result Communication – Creating clear charts for reports and presentations.

If you are using Anaconda, Matplotlib is already installed. Otherwise, you can install it using:

```
pip install matplotlib
```

To import Matplotlib library:

```
import matplotlib.pyplot as plt
```

Here, pyplot is a submodule that provides functions to create plots and plt is the alias.

To plot a simple line

```
#CODE: Line plot

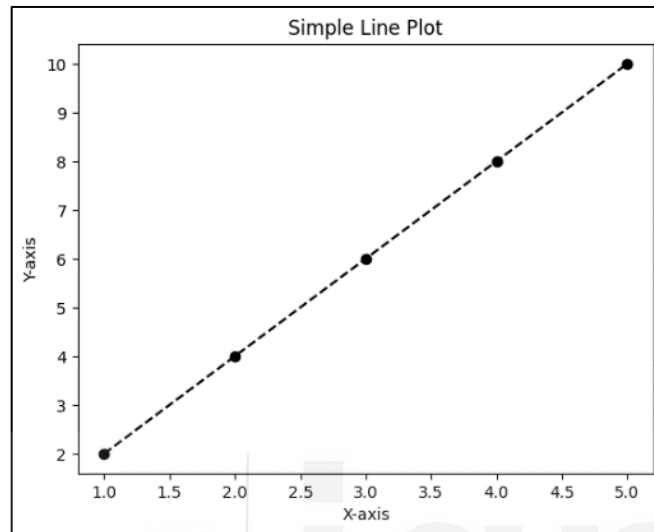
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]    #x-axis : #horizontal axis
                      (independent variable)
y = [2, 4, 6, 8, 10]  #y-axis : vertical axis
                      (dependent variable)

plt.plot(x, y, color='black', linestyle='--',
marker='o')
#color - sets line color
#linestyle - changes line style (--, -, :)
#marker - marks data points (o, s, *)
```

```
plt.title("Simple Line Plot") #plot title
plt.xlabel("X-axis") #axis labels
plt.ylabel("Y-axis") #axis labels
plt.show()
```

OUTPUT:



Bar chart can be plotted as follows:

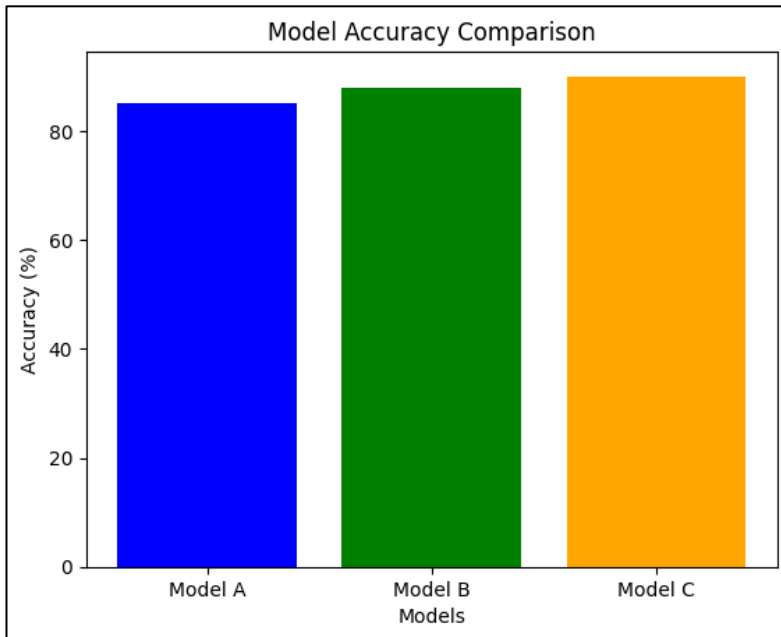
```
#CODE: Bar Chart

import matplotlib.pyplot as plt

models = ['Model A', 'Model B', 'Model C']
accuracy = [85, 88, 90]

#bar function is used for bar chart
plt.bar(models, accuracy,
color=['blue', 'green', 'orange'])
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")
plt.show()
```

OUTPUT:

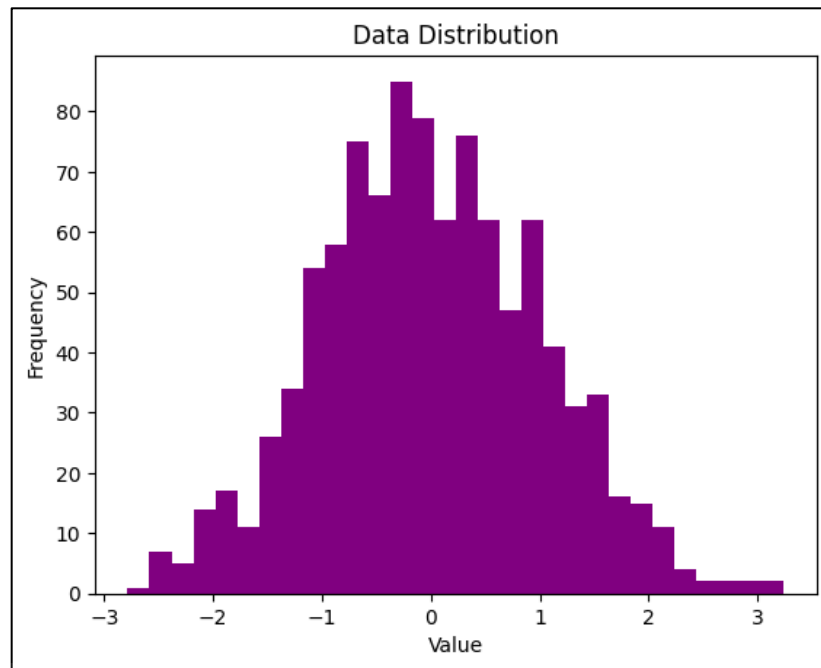


Histograms are used to show data distribution. Code is as follows:

```
#CODE: Histogram

import matplotlib.pyplot as plt
import numpy as np
data = np.random.randn(1000)

plt.hist(data, bins=30, color='purple') #hist
function is used for histogram
plt.title("Data Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



Scatter Plot shows relationship between two variables. It is widely used in AI to study correlations.

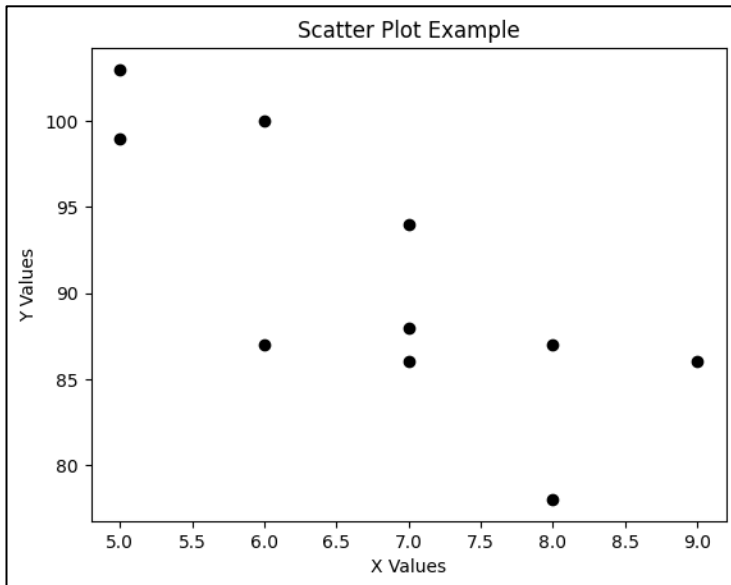
```
#CODE: Scatter Plot

import matplotlib.pyplot as plt

x = [5,7,8,7,6,9,5,6,7,8]
y = [99,86,87,88,100,86,103,87,94,78]

plt.scatter(x, y, color='black')
plt.title("Scatter Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show
```

OUTPUT:



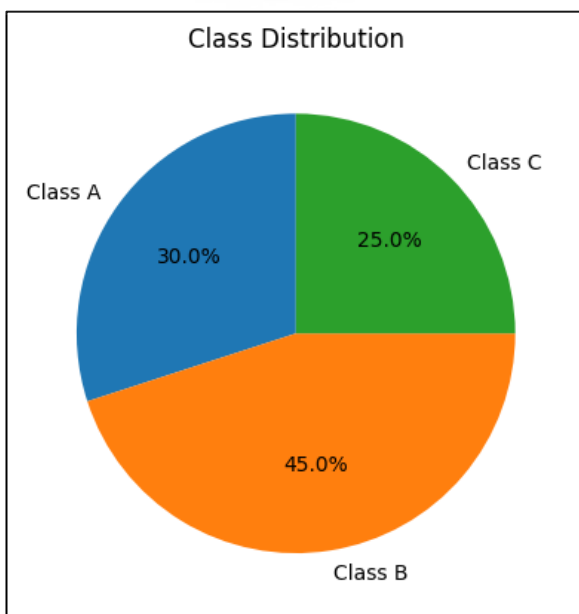
To represent proportions, Pie Charts are used as follows:

```
#CODE: Pie Chart

import matplotlib.pyplot as plt

labels = ['Class A', 'Class B', 'Class C']
sizes = [30, 45, 25]
plt.figure()
plt.pie(sizes, labels=labels, autopct='%1.1f%%',
startangle=90)
plt.title("Class Distribution")
plt.show()
```

Output:



We can display multiple plots together using subplots. They are useful in AI to visualize different aspects of the same dataset together.

```
#CODE: Subplots

import matplotlib.pyplot as plt

fig, axs = plt.subplots(2, 2) # set the frame for 4
plots arranged as 2 X 2

x = [1, 2, 3, 4, 5]      #x-axis : #horizontal axis
                          #independent variable)
y = [2, 4, 6, 8, 10]     #y-axis : vertical axis
                          #(dependent variable)

axs[0, 0].plot(x, y)
axs[0, 0].set_title("Line Plot")

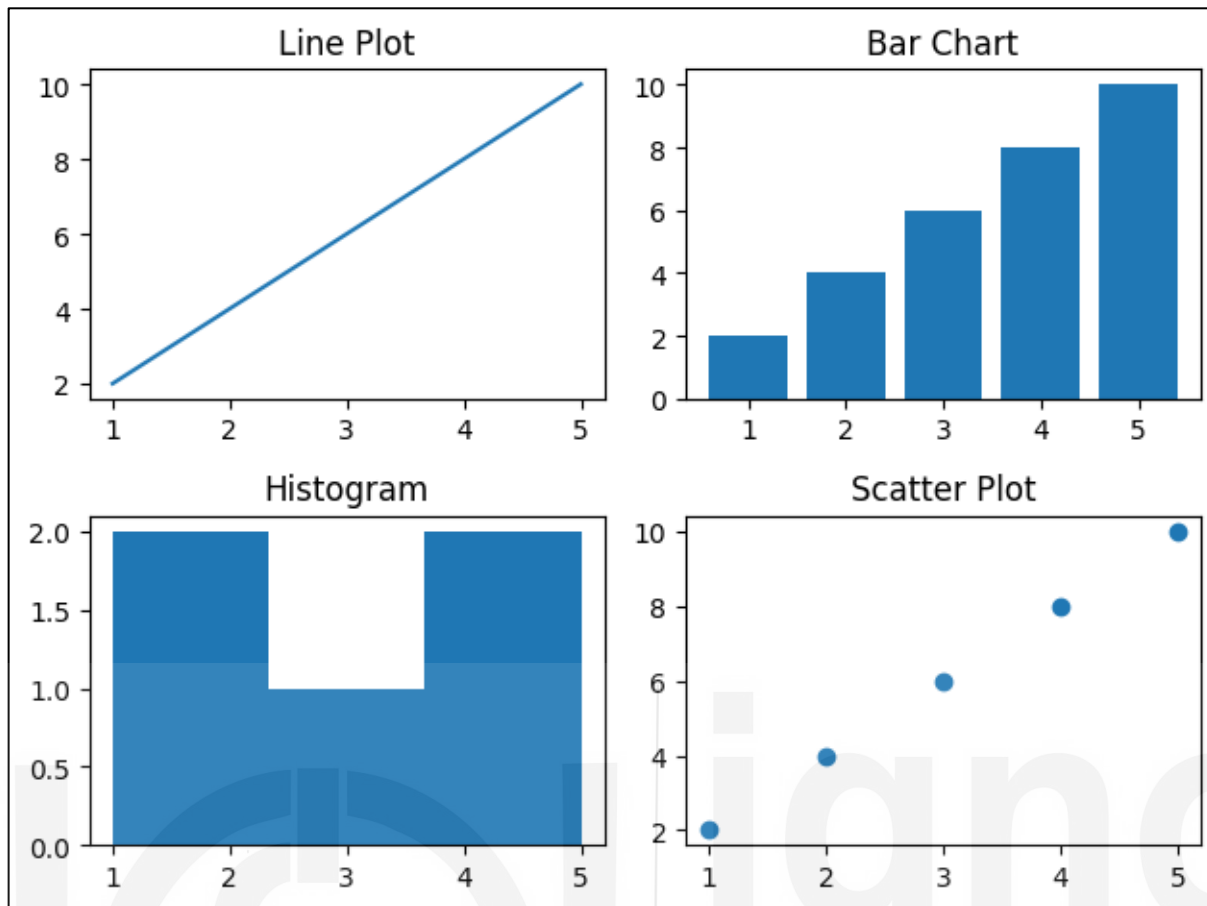
axs[0, 1].bar(x,y)
axs[0, 1].set_title("Bar Chart")

axs[1, 0].hist(x, bins=3)
axs[1, 0].set_title("Histogram")

axs[1, 1].scatter(x, y)
axs[1, 1].set_title("Scatter Plot")

plt.tight_layout()
plt.show()
```

OUTPUT;



Plots created using above codes can be saved for documentation in different formats: PNG, JPG, PDF, SVG. Code to save the plot is

```
#CODE: Save plot
plt.savefig("plot.png")
```

Matplotlib is a highly flexible visualization library that allows users to create almost any type of chart or graph, making it extremely useful for AI research and the data analysis. It can be integrated with popular Python libraries such as NumPy, Pandas, TensorFlow and PyTorch which enables the efficient handling and visualization of the complex datasets. Moreover, Matplotlib offers extensive the customization options. It allows users to modify titles, labels, styles and legends to enhance the clarity and presentation.

Despite its advantages, Matplotlib can be somewhat challenging for beginners due to its relatively complex syntax. Creating detailed or specific plots often requires writing multiple lines of code which can make the process time-consuming.

11.3.4 Scikit-learn

AI and ML are all about making computers learn from data. To do this, we need tools that make model building simple and efficient. One of the most popular and beginner-friendly libraries in Python for ML is Scikit-learn (also known as

sklearn). It is an open-source library built on top of NumPy, Pandas and Matplotlib. It provides simple and consistent functions to preprocess data, build machine learning models, evaluate performance and fine-tune results.

If you are using Anaconda, Scikit-learn is already installed. Otherwise, you can install it using:

```
pip install scikit-learn
```

To import Scikit-learn library:

```
import sklearn
```

Most of the time, we import specific modules like:

```
from sklearn.model_selection import  
train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import accuracy_score
```

Every machine learning task in Scikit-learn usually follows these steps:

1. Load Data
2. Split Data into training and testing sets
3. Preprocess Data (cleaning, scaling, encoding)
4. Choose Model (e.g., Linear Regression, Decision Tree)
5. Train Model using training data
6. Evaluate Model using testing data
7. Fine-tune if necessary

Code for Linear Regression (Supervised Learning) to predicts continuous values is as follows:

```
#CODE: Linear Regression  
  
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
  
# Sample data  
data = {  
    'Hours_Studied': [1,2,3,4,5,6,7,8],  
    'Marks': [30,35,50,55,65,70,75,85]  
}  
df = pd.DataFrame(data)  
  
# Features and target  
X = df[['Hours_Studied']] # Independent variable  
y = df['Marks']           # Target variable
```

```

# Train-test split
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("Predicted Marks:", y_pred)

```

This Python code demonstrates a simple example of Linear Regression using the scikit-learn library. It first creates a sample dataset containing two columns: Hours_Studied (independent variable) and Marks (dependent variable). The dataset is then converted into a pandas DataFrame. The feature (X) is defined as the number of study hours and the target (y) is the corresponding marks. The data is split into training and testing sets using train_test_split(), where 80% is used for training and 20% for testing. A LinearRegression model is then created and trained on the training data using the fit() method. After training, the model predicts marks for the test data (X_test) and the predicted values (y_pred) are printed, showing how well the model estimates marks based on hours studied.

OUTPUT:

```
Predicted Marks: [54.16666667 85.5952381 ]
```

Code for classification with Decision Tree to predicts categories as follows:

```

#CODE: Decison Tree

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Sample data
data = {
    'Age': [18, 19, 20, 21, 22, 23],
    'Score': [55, 60, 65, 70, 75, 80],
    'Passed': [0, 0, 0, 1, 1, 1]
}
df = pd.DataFrame(data)

X = df[['Age', 'Score']]
y = df['Passed']

# Train-test split

```

```

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.3)

# Train model
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

```

This Python code demonstrates the use of a Decision Tree Classifier for predicting whether a student has passed or not based on their Age and Score. It begins by creating a small dataset with three columns: Age, Score and Passed (where 1 indicates pass and 0 indicates fail) and stores it in a pandas DataFrame. The features (X) are taken as Age and Score, while the target variable (y) is Passed. The dataset is split into training and testing sets using `train_test_split()`, with 70% used for training and 30% for testing. A `DecisionTreeClassifier` model is then created and trained on the training data using the `fit()` method. After training, the model predicts outcomes for the test data (X_test) using `predict()`. Finally, the accuracy of the model is calculated using `accuracy_score()` and printed, showing how well the decision tree predicts student pass/fail outcomes.

Code for clustering with K-Means to group similar data as follows:

```

#CODE: Clustering : K-Means

import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Random dataset
X =
np.array([[1,2],[1,4],[1,0],[10,2],[10,4],[10,0]])

# Train K-Means model
kmeans = KMeans(n_clusters=2, random_state=0).fit(X)

print("Cluster Centers:", kmeans.cluster_centers_)

# Plot
plt.scatter(X[:,0], X[:,1], c=kmeans.labels_)
plt.scatter(kmeans.cluster_centers_[:,0],
kmeans.cluster_centers_[:,1], marker='x',
color='red')
plt.show()

```

This Python code illustrates K-Means clustering, an unsupervised learning technique used to group similar data points. It begins by creating a small random dataset X containing six two-dimensional points. The KMeans algorithm from scikit-learn is then used to divide the data into two clusters (n_clusters=2). The model is trained using the fit() method, which calculates the optimal positions of the cluster centers. The coordinates of these centers are printed using kmeans.cluster_centers_. Finally, the data points are visualized with matplotlib, where each point is colored according to its assigned cluster (kmeans.labels_) and the cluster centers are marked with red “X” symbols, clearly showing the two formed groups.

Choosing the right model settings (hyperparameters) in ML model improves the accuracy. Sklearn provides a function to search for best parameters.

```
#CODE: Best parameter search

from sklearn.model_selection import GridSearchCV

params = {'max_depth':[2,3,4]}
clf = DecisionTreeClassifier()
grid = GridSearchCV(clf, params, cv=3)
grid.fit(X, y)

print("Best Parameters:", grid.best_params_)
```

This Python code shows how to perform hyperparameter tuning using GridSearchCV to find the optimal configuration for a machine learning model. Here, a DecisionTreeClassifier is created and a parameter grid params is defined with different values of max_depth (2, 3 and 4), which controls how deep the decision tree can grow. The GridSearchCV function from scikit-learn runs the model multiple times—once for each combination of parameters—using 3-fold cross-validation (cv=3) to evaluate model performance. It then fits the model on the dataset (X, y) and automatically selects the parameter value that produces the best results. Finally, grid.best_params_ prints the best parameter setting (in this case, the most suitable max_depth) for achieving the highest model accuracy.

Scikit-learn is a beginner-friendly library with simple and easy-to-understand syntax. It is suitable for newcomers to machine learning. It is a comprehensive library that offers almost all classical machine learning algorithms such as regression, classification, clustering and model selection. The library is reliable and widely used for both academia and industry for research and application development. It can be integrated with other popular Python libraries like NumPy, Pandas and various visualization tools.

Despite its strengths, Scikit-learn is not designed for deep learning applications, where frameworks like TensorFlow or PyTorch are preferred. It works best for small to medium-sized datasets while handling very large datasets which may require distributed or GPU-based systems like Spark ML. Additionally, Scikit-learn has limited support for online or real-time learning which makes it less suitable for scenarios requiring continuous model updates.

11.3.5. TensorFlow

TensorFlow is an open-source machine learning and deep learning framework developed by Google Brain in 2015. It is one of the most widely used libraries for building and training AI models. The name “TensorFlow” comes from the concept of tensors (multidimensional data arrays) that flow through a system of mathematical operations.

In TensorFlow, data is represented as tensors. A tensor is a generalization of vectors and matrices to higher dimensions.

- A scalar (a single number) is a 0-D tensor.
- A vector (a list of numbers) is a 1-D tensor.
- A matrix (rows and columns) is a 2-D tensor.
- Higher dimensions are possible for more complex data.

Example to creates a 2x2 matrix (a rank-2 tensor)

```
#CODE: Tenser Flow Example

import tensorflow as tf
x = tf.constant([[1, 2], [3, 4]])
print(x)
```

OUTPUT:

```
tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
```

TensorFlow uses the idea of a computation graph to represent mathematical operations. In a computation graph, nodes represent operations (like addition, multiplication) and edges represent the tensors (data) that flow between operations.

In early versions of TensorFlow (before version 2.0), users had to first define the graph and then run it in a session. This was called a static graph, which was powerful but difficult to debug. From TensorFlow 2.0 onward, TensorFlow introduced eager execution, which means operations run immediately as they are called — similar to how Python works. This made TensorFlow much easier and more intuitive, like PyTorch.

TensorFlow provides a high-level API called Keras to make building neural networks simple and user-friendly. A neural network is made up of layers — such as input, hidden and output layers — that process data step by step to make predictions.

Example:

```
#CODE: Neural Network Example
```

```

from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation='relu',
input_shape=(10,)),
    layers.Dense(1, activation='sigmoid')
])

model.summary()

```

OUTPUT:

Model: "sequential_3"

Layer (type) Param #	Output Shape
dense_6 (Dense) 176	(None, 16)
dense_7 (Dense) 17	(None, 1)
Total params: 193 (772.00 B)	
Trainable params: 193 (772.00 B)	
Non-trainable params: 0 (0.00 B)	

Here, Sequential creates a linear stack of layers. The Dense layer is a fully connected layer used in many neural networks. TensorFlow automatically handles complex computations like backpropagation and gradient descent during training.

To compile and train the model:

```

model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10)

```

11.3.6 PyTorch

After TensorFlow, another powerful and easy-to-use deep learning framework was developed: PyTorch, created by Facebook's AI Research (FAIR) team in 2016. Like TensorFlow, PyTorch is used for building, training and deploying deep learning models. However, PyTorch focuses more on simplicity, flexibility

and dynamic computation, which makes it very popular among researchers, students and developers.

TensorFlow (especially in its early versions) used static computation graphs, meaning you had to define the entire model before running it. PyTorch, on the other hand, uses a dynamic computation graph that means it builds the graph as you go. This makes PyTorch feel more like regular Python code, which is easier to write, understand and debug.

The basic building block in PyTorch is the tensor, similar to NumPy arrays but faster and compatible with GPUs. Tensors are used to store and manipulate data.

```
#CODE: Pytorch Example

import torch
x = torch.tensor([[1, 2], [3, 4]])
print(x)
print(x * 2)    # Tensor operations
```

OUTPUT:

```
tensor([[1, 2],
        [3, 4]])
tensor([[2, 4],
        [6, 8]])
```

PyTorch provides autograd, a feature that automatically calculates gradients (derivatives) for optimization. It is mainly used in training neural networks through backpropagation.

```
#CODE: Pytorch Example 2

x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 4 * x
y.backward()    # Compute gradient
print(x.grad)   # dy/dx = 3x2 + 4
```

Here, PyTorch automatically computes the gradient of y with respect to x , which is very useful for updating model parameters during training.

PyTorch provides a powerful module called `torch.nn` that helps define and train neural networks easily. It includes layers, activation functions and loss functions.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 3),    # Input layer: 2 neurons →
    nn.ReLU(),          # Activation function
    Hidden layer: 3 neurons
```

```

nn.Linear(3, 1)      # Output layer
)
print(model)

```

Check your Progress 1

1. Write Python program using matplotlib to draw a pie chart for your marks in 5 subjects.

.....

.....

.....

2. Write a python code to do statistical analysis (average, min, max, standard deviation) of your marks as mentioned in previous question.

.....

.....

.....

3. What are tensors and how they help us to write program for deep leaning?

.....

.....

.....

11.4 HANDS ON IMPLEMENTATION

Following section discusses the Python implementation of various well-known problems in AI-domain such as: N-Queen Problem, Min-Max Algorithm and Alpha-Beta Pruning, Breadth First Search, Depth First Search, Best First Search, A* Algorithm, AO* Algorithm and Iterative Deepening A*.

11.4.1 N-Queen Problem

The N-Queen problem is a classic backtracking problem where the goal is to place N queens on an N×N chessboard such that no two queens attack each other. Since queens in chess can attack along rows, columns and diagonals, the challenge is to arrange them in a safe way. The Python program solves this by using a method called backtracking. This means it tries to place a queen in the first row, checks if it is safe and then moves to the next row. If it finds a row where no safe place is available, it goes back (backtracks) and changes the position of the previous queen. This process continues until all queens are placed correctly. For more details about N-queen problem, refer to section 2.4.

Following is the code to solve N-queen problem:

```

#CODE; N-Queen Problem

```

```

def print_solution(board): #function to print the
solution
    n = len(board)
    for i in range(n):
        row = ""
        for j in range(n):
            if board[i][j] == 1:
                row += "Q "
            else:
                row += ". "
        print(row)
    print("\n")

def is_safe(board, row, col):
#Function to check if a particular position is safe
to place the next queen
    n = len(board)

    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i, j = row, col
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True

def solve_nqueens(board, row): #Function to solve N-
queen problem
    n = len(board)
    if row == n: # All queens are placed
        print_solution(board)
        return True

    res = False
    for col in range(n):

```

```

        if is_safe(board, row, col):
            board[row][col] = 1 # Place queen
            res = solve_nqueens(board, row + 1) or
res #recursive function call
            board[row][col] = 0 # Backtrack

    return res

def n_queens(n):
    board = [[0] * n for _ in range(n)]
    if not solve_nqueens(board, 0):
        print("No solution exists")

# Example: Solve for N=4
n_queens(4)

```

OUTPUT:

```

. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .

```

Here, Q represents a queen and . represents an empty cell. It prints **all possible solutions** for the N-Queen problem.

In the Python code, N-Queen problem is solved using recursion. For each row, the program tries placing a queen in every column and before placing, it checks whether the position is safe. The `is_safe` function ensures that there is no queen in the same column, or along the left and right diagonals above the current row. If the position is safe, the queen is placed and the algorithm recursively moves to the next row. If at any point no safe position is found, it backtracks by removing the previously placed queen and trying another column. Once all queens are successfully placed, the configuration is stored as a valid solution. After finding solutions, the code prints the total number of them and visualizes each one on a chessboard. For example, when $n=4$, the program finds 2 valid solutions and displays both boards with queens arranged so that none can attack each other.

11.4.2 Min-Max Algorithm and Alpha-Beta Pruning

The Min-Max algorithm is a decision-making algorithm used in two-player games where one player tries to maximize the score (Maximizer) while the other tries to minimize it (Minimizer). The idea is that both players play optimally. The algorithm builds a game tree of all possible moves. At each step, if it is the

Maximizer's turn, the algorithm chooses the move with the highest possible value, assuming the opponent will respond in the best way to minimize the score. On the other hand, if it is the Minimizer's turn, it chooses the move with the lowest possible value, assuming the opponent will try to maximize their advantage later. This process continues until a terminal state is reached (win, lose, or draw) and values are assigned to the outcomes. The values then backtrack up the tree and the root player picks the move that guarantees the best result against an optimal opponent. For example, in tic-tac-toe, the Min-Max algorithm can simulate all possible moves and ensure that the AI either wins or forces a draw, but never loses. Refer to Section 2.6 for more details about Min-Max algorithm. Here is the code to play tic-tac-toe using Min-Max approach.

```
#CODE: Min-Max Algorithm for Tic-Tac-Toe Game

import math

# Check if there are moves left on the board
def is_moves_left(board):
    for row in board:
        if "_" in row:
            return True
    return False

# Evaluate the board
def evaluate(board):
    # Check rows
    for row in board:
        if row.count("X") == 3:
            return 10
        elif row.count("O") == 3:
            return -10

    # Check columns
    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col]:
            if board[0][col] == "X":
                return 10
            elif board[0][col] == "O":
                return -10

    # Check diagonals
    if board[0][0] == board[1][1] == board[2][2]:
        if board[0][0] == "X":
            return 10
        elif board[0][0] == "O":
            return -10
    if board[0][2] == board[1][1] == board[2][0]:
        if board[0][2] == "X":
            return 10
        elif board[0][2] == "O":
```

```

        return -10

    return 0

# Minimax function
def minimax(board, depth, is_max):
    score = evaluate(board)

    # If Maximizer has won
    if score == 10:
        return score - depth # prefer faster wins

    # If Minimizer has won
    if score == -10:
        return score + depth # prefer slower losses

    # If no moves left => draw
    if not is_moves_left(board):
        return 0

    if is_max:
        best = -math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == "_":
                    board[i][j] = "X"
                    best = max(best, minimax(board,
depth + 1, False))
                    board[i][j] = "_"
            return best
        else:
            best = math.inf
            for i in range(3):
                for j in range(3):
                    if board[i][j] == "_":
                        board[i][j] = "O"
                        best = min(best, minimax(board,
depth + 1, True))
                        board[i][j] = "_"
            return best

# Find the best move
def find_best_move(board):
    best_val = -math.inf
    best_move = (-1, -1)

    for i in range(3):
        for j in range(3):
            if board[i][j] == "_":
                board[i][j] = "X"
                move_val = minimax(board, 0, False)

```

```

        board[i][j] = "_"
        if move_val > best_val:
            best_move = (i, j)
            best_val = move_val

    return best_move

# Example Board
board = [
    ["X", "O", "X"],
    ["O", "O", "_"],
    ["_", "_", "_"]
]

best_move = find_best_move(board)
print("The AI should place at:", best_move)

```

This Python program uses the Min-Max algorithm to decide the best move for the AI in Tic-Tac-Toe. The board is represented as a 3×3 list, where "X" is the AI, "O" is the opponent and "_" represents an empty cell. The evaluate function assigns a score to the board: +10 if the AI wins, -10 if the opponent wins and 0 for a draw. The minimax function then simulates all possible future moves. If it is the AI's turn (Maximizer), it tries to maximize the score by choosing the move that leads to the best outcome. If it is the opponent's turn (Minimizer), it chooses moves that minimize the score, making the game harder for the AI. To make decisions smarter, the depth of the recursion is included in the score, so the AI prefers to win quickly and delay losses as much as possible. Finally, the find_best_move function loops through all available empty spaces, uses minimax to evaluate them and picks the move with the highest score. When we run the program on a given board, it prints the best position for the AI to play next. This ensures the AI always plays optimally: it will win when possible, or at least force a draw.

While the algorithm guarantees the best result, it can be slow for large games because it explores all possible moves. To make it efficient, a technique called Alpha-Beta Pruning is often added to cut off unnecessary branches in the game tree. It is explained in detailed in Section 2.7.

```

#CODE: Alpha-beta Pruning for Tic-Tac-Toe Game

import math

# Check if moves left
def is_moves_left(board):
    for row in board:
        if "_" in row:
            return True
    return False

# Evaluate the board
def evaluate(board):

```

```

# Check rows
for row in board:
    if row.count("X") == 3:
        return 10
    elif row.count("O") == 3:
        return -10

# Check columns
for col in range(3):
    if board[0][col] == board[1][col] ==
board[2][col]:
        if board[0][col] == "X":
            return 10
        elif board[0][col] == "O":
            return -10

# Check diagonals
if board[0][0] == board[1][1] == board[2][2]:
    if board[0][0] == "X":
        return 10
    elif board[0][0] == "O":
        return -10
if board[0][2] == board[1][1] == board[2][0]:
    if board[0][2] == "X":
        return 10
    elif board[0][2] == "O":
        return -10

return 0

# Minimax with Alpha-Beta Pruning
def minimax(board, depth, alpha, beta, is_max):
    score = evaluate(board)

    # If terminal state
    if score == 10:
        return score - depth
    if score == -10:
        return score + depth
    if not is_moves_left(board):
        return 0

    if is_max: # Maximizer (AI "X")
        best = -math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == "_":
                    board[i][j] = "X"
                    val = minimax(board, depth + 1,
alpha, beta, False)
                    board[i][j] = "_"

```

```

        best = max(best, val)
        alpha = max(alpha, best) #
Setting the value for alpha
        if beta <= alpha: # Prune
branch
            break

        return best
    else: # Minimizer (Opponent "O")
        best = math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == "_":
                    board[i][j] = "O"
                    val = minimax(board, depth + 1,
alpha, beta, True)
                    board[i][j] = "_"
                    best = min(best, val)
                    beta = min(beta, best) # Setting
the value for beta
                if beta <= alpha: # Prune
branch
                    break

            return best

# Find the best move
def find_best_move(board):
    best_val = -math.inf
    best_move = (-1, -1)

    for i in range(3):
        for j in range(3):
            if board[i][j] == "_":
                board[i][j] = "X"
                move_val = minimax(board, 0, -
math.inf, math.inf, False)
                board[i][j] = "_"
                if move_val > best_val:
                    best_move = (i, j)
                    best_val = move_val

    return best_move

# Example Board
board = [
    ["X", "O", "X"],
    ["O", "O", "_"],
    ["_", "_", "_"]
]

best_move = find_best_move(board)
print("The AI should place at:", best_move)

```

This program uses the Min-Max algorithm with Alpha-Beta Pruning to play Tic-Tac-Toe optimally. The AI is "X" (Maximizer) and the opponent is "O" (Minimizer). The evaluate function scores a board: +10 if AI wins, -10 if opponent wins and 0 if it's a draw. The minimax function explores possible moves recursively: the AI tries to maximize the score, while the opponent tries to minimize it. Alpha (best already guaranteed for Maximizer) and Beta (best already guaranteed for Minimizer) are used to prune (cut off) unnecessary branches of the game tree. If a branch cannot possibly improve the final decision, the algorithm skips it, saving time. The find_best_move function checks all empty spots, runs minimax with pruning and picks the move with the highest value. The result is that the AI always makes the optimal move, but faster than regular Min-Max, since many losing branches are ignored.

Check your Progress 2

1. What is the purpose of the is_safe() function in the N-Queen implementation?

.....

.....

.....

2. In the backtracking solution of the N-Queen problem, what happens when a position is not safe?

.....

.....

.....

3. In Minimax implementation, what is the base case for recursion?

.....

.....

.....

11.4.3 Breadth First Search

Breadth-First Search (BFS) (explained in Section 3.3.1) is a graph traversal algorithm that explores nodes level by level. Refer to the graph given in figure 1, BFS starts from a given node (here "A") and visits all its direct neighbors before moving on to their neighbors. In the program, a queue is used to ensure nodes are processed in the correct order. First, "A" is added to the queue. Then, its neighbors "B" and "C" are visited and enqueued. Next, BFS explores "B", which leads to "D" and "E". Then it processes "C", leading to "F". Finally, "D", "E" and "F" are visited, but since "F" was already added earlier, it is not revisited. The traversal order becomes ['A', 'B', 'C', 'D', 'E', 'F']. BFS is very useful for finding the shortest path in an unweighted graph, or for exploring networks like social connections, web crawlers and AI game moves.

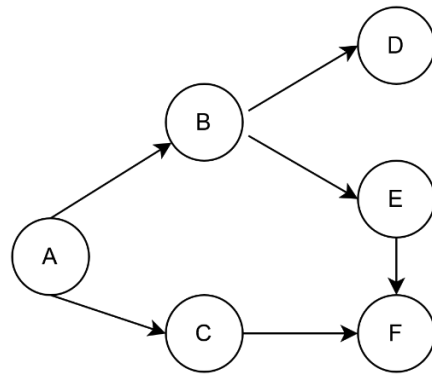


Figure 1: Sample graph

```

#CODE: BFS

from collections import deque

# BFS function
def bfs(graph, start):
    visited = set() # To keep track of
    visited nodes
    queue = deque([start]) # Queue for BFS
    order = [] # To store traversal
    order

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            order.append(node)

            # Add neighbors to queue
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

    return order

# Example Graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Run BFS
result = bfs(graph, 'A')
print("BFS Traversal:", result)

```

OUTPUT:

```
BFS Traversal: ['A', 'B', 'C', 'D', 'E', 'F']0
```

11.4.4 Depth First Search

Depth-First Search (DFS) (explained in Section 3.3.4) is a graph traversal algorithm that explores as far as possible along one path before backtracking. In this program, DFS (refer figure 1) starts from node "A". First, it visits "A", then moves to its first neighbor "B". From "B", it goes to "D", which has no further neighbors, so it backtracks. Next, it visits "E" (another neighbor of "B") and from "E", it proceeds to "F". Once "F" is done, DFS backtracks again to explore any remaining nodes. Finally, it visits "C" (the other neighbor of "A"), but since "F" was already visited, it does not repeat it. The traversal order is ['A', 'B', 'D', 'E', 'F', 'C']. Unlike BFS, which explores level by level, DFS goes deep into the graph before backtracking, making it useful for tasks like maze solving, pathfinding and topological sorting.

```
#CODE: DFS

# DFS function using recursion
def dfs(graph, node, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    if node not in visited:
        visited.add(node)
        order.append(node)

        # Visit all neighbors
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited, order)

    return order

# Example Graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Run DFS
result = dfs(graph, 'A')
print("DFS Traversal:", result)
```

OUTPUT:

```
DFS Traversal: ['A', 'B', 'D', 'E', 'F', 'C']
```

11.4.5 Best First Search

Best First Search (explained in Section 3.7.3) is a graph search algorithm that uses a heuristic value (represented by h) to decide which node to explore first. Heuristic value is an estimate of closeness of the current node to the goal. Unlike normal BFS or DFS, it doesn't explore blindly; instead, it chooses the path that looks most promising based on the heuristic value. In this program, we use a priority queue to always expand the node with the smallest heuristic value. Refer to figure 2, consider start node as A and goal node as F. Starting from "A", the algorithm looks at neighbors "B" ($h=4$) and "C" ($h=3$). Since "C" has a lower heuristic, it is explored first. From "C", we reach "F" directly. Since "F" is the goal, the search stops. The final path is ['A', 'C', 'F']. This makes Best First Search faster than normal BFS or DFS when a good heuristic is available, though it does not always guarantee the shortest path (unlike A*). It is often used in AI pathfinding and puzzle-solving where an informed guess can save a lot of time.

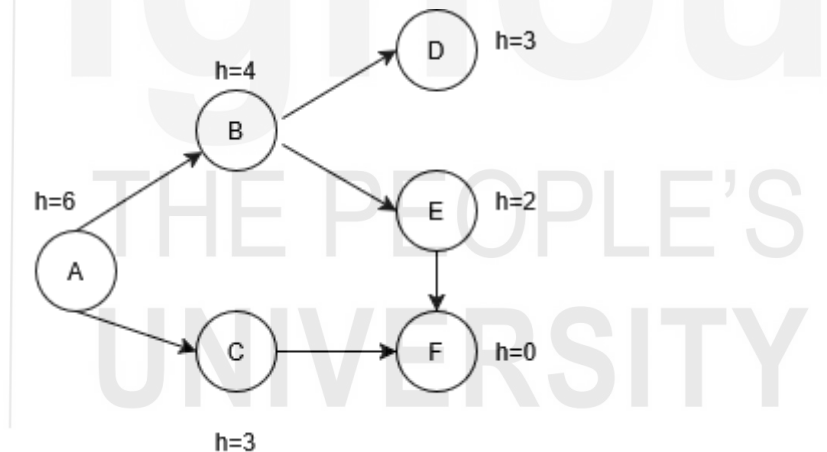


Figure 2: Sample graph for Best First Search

```
#CODE: Best First Search

import heapq

# Best First Search Algorithm
def best_first_search(graph, heuristic, start, goal):
    visited = set()
    pq = [] # priority queue (min-heap)
    heapq.heappush(pq, (heuristic[start], start)) # (heuristic, node)
    path = []
```

```

while pq:
    heuristic, node = heapq.heappop(pq)

    if node not in visited:
        visited.add(node)
        path.append(node)

        if node == goal:
            return path

        for neighbor in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq,
(heuristic[neighbor], neighbor))

    return path

# Example Graph with heuristic values
# Example Graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 3,
    'D': 5,
    'E': 2,
    'F': 0    # Goal node
}

# Run Best First Search
start, goal = 'A', 'F'
result = best_first_search(graph, heuristic, start,
goal)
print(f"Best First Search Path from {start} to
{goal}:", result)

```

OUTPUT:

```
Best First Search Path from A to F: ['A', 'C', 'F']
```

11.4.6 A* Algorithm

A* Search (explained in section 3.8) is one of the most powerful and widely used search algorithms in AI because it always finds the shortest path if the heuristic is correct. It combines the strengths of Dijkstra's Algorithm and Best First Search. In this program, each node keeps two values:

- $g(n)$ = actual cost from the start node to the current node.
- $h(n)$ = heuristic estimate from the current node to the goal.

Finally, total estimated cost is calculated as $f(n) = g(n) + h(n)$

The algorithm uses a priority queue to always expand the node with the lowest $f(n)$. Starting at "A", it explores "B" ($f(n) = 4+2= 6$) and "C" ($f(n) = 3+1= 4$). It selects C as it has a lower overall $f(n)$ because of its heuristic. From "C", the goal "F" is quickly reached, giving the optimal path ['A', 'C', 'F']. This shows how A* balances between exploring the cheapest path so far ($g(n)$) and the most promising direction ($h(n)$), making it efficient and optimal. A* is widely used in maps (Google Maps, GPS routing), robotics and puzzle-solving because it guarantees the shortest path when the heuristic is admissible.

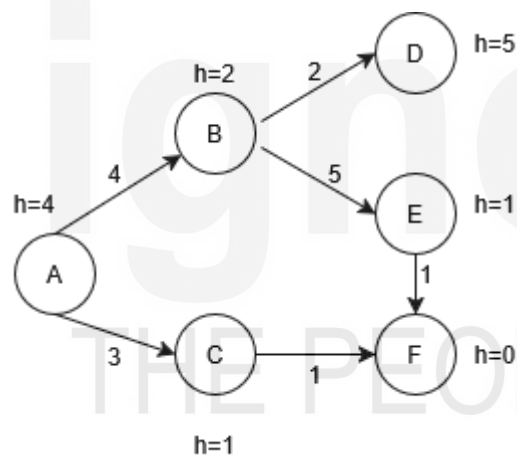


Figure 3: Sample graph for A* algorithm

```

#CODE: A* Search

import heapq

# A* Search Algorithm
def a_star_search(graph, heuristics, start, goal):
    pq = [] # priority queue (min-heap)
    heapq.heappush(pq, (0, start)) # (f = g+h,
node)

    parent = {start: None} # to reconstruct path
    g_cost = {start: 0}    # cost from start to
current node

    while pq:
        f, node = heapq.heappop(pq)
  
```

```

        if node == goal:
            # reconstruct path
            path = []
            while node:
                path.append(node)
                node = parent[node]
            return path[::-1]

    for neighbor, cost in graph[node]:
        tentative_g = g_cost[node] + cost

        if neighbor not in g_cost or tentative_g
< g_cost[neighbor]:
            g_cost[neighbor] = tentative_g
            f_cost = tentative_g +
heuristics[neighbor]
            heapq.heappush(pq, (f_cost,
neighbor))
            parent[neighbor] = node

    return None

# Example Graph (with edge costs)
graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 1)],
    'D': [],
    'E': [('F', 1)],
    'F': []
}

# Heuristic values (straight-line estimate to goal
'F')
heuristics = {
    'A': 4,
    'B': 2,
    'C': 1,
    'D': 5,
    'E': 1,
    'F': 0
}

# Run A* Search
start, goal = 'A', 'F'
result = a_star_search(graph, heuristics, start,
goal)
print(f"A* Path from {start} to {goal}:", result)

```

OUTPUT:

11.4.7 AO* Algorithm

The AO* (And-Or Star) algorithm (explained in Section 3.9) is used in AND-OR graphs (as shown in Figure 4) for problem solving. Unlike A* or BFS/DFS, which work on simple paths, AO* works where solutions require satisfying multiple subgoals together (AND nodes) or choosing between alternatives (OR nodes).

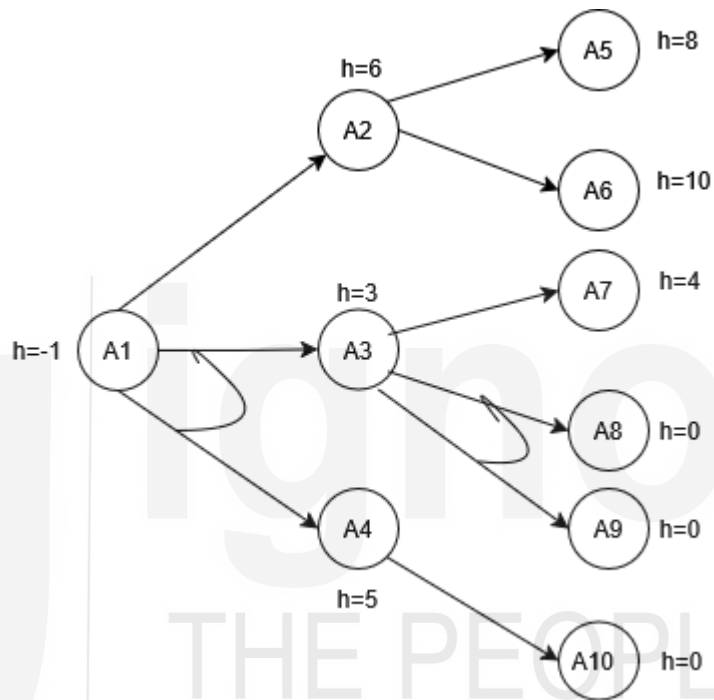


Figure 4: Sample graph for AO* algorithm

```
#CODE: AO* Search
```

```
#function to calculate the cost of each node
```

```
def cal_cost(H, cd, wt = 1):
```

```
    cost = {}
```

```
    if 'AND' in cd:
```

```
        nodes1 = cd['AND']
```

```
        andpath = ' AND '.join(nodes1)
```

```
        finalandpath = sum(H[node]+wt for node in
nodes1)
```

```
        cost[andpath] = finalandpath
```

```
    if 'OR' in cd:
```

```
        nodes2 = cd['OR']
```

```
        orpath = ' OR '.join(nodes2)
```

```
        finalorpath = min(H[node]+wt for node in
nodes2)
```

```
        cost[orpath] = finalorpath
```

```
    return cost
```

```

# function for cost updation
def update_cost(H, graph, wt=1):
    nodes = list(graph.keys())
    nodes.reverse()
    mincost = {}
    for key in nodes:
        cd = graph[key]
        print(key, ': ', graph[key], '->',
cal_cost(H, cd, wt))
        c = cal_cost(H, cd, wt)
        H[key] = min(c.values())
        mincost[key] = cal_cost(H, cd, wt)
    return mincost

# function to print the shortest path
def shortest_path(start, Updated_cost, H):
    path = start
    if start in Updated_cost.keys():
        Min_cost = min(Updated_cost[start].values())
        key = list(Updated_cost[start].keys())
        values = list(Updated_cost[start].values())
        Index = values.index(Min_cost)

        nn = key[Index].split()

        if len(nn) == 1:
            start = nn[0]
            path += '<<-' + shortest_path(start,
Updated_cost, H)
        else:
            path += '<<-( '+key[Index]+' ) '
            start = nn[0]
            path += '[' + shortest_path(start,
Updated_cost, H) + ' + '
            start = nn[-1]
            path += shortest_path(start,
Updated_cost, H) + ']'

    return path

# Sample
H = {'A1': -1, 'A2': 6, 'A3': 3, 'A4': 5, 'A5': 8,
'A6': 10, 'A7': 4, 'A8': 0, 'A9': 0, 'A10': 0}

graph = {
    'A1': {'OR': ['A2'], 'AND': ['A3', 'A4']},
    'A2': {'OR': ['A5', 'A6']},
    'A3': {'OR': ['A7'], 'AND': ['A8', 'A9']},
    'A4': {'OR': ['A10']}

```

```

}

# Weight of each edge is considered as 1
wt = 1

Updated_cost = update_cost(H, graph, wt)
print('Shortest Path:\n', shortest_path('A1',
Updated_cost, H))

```

OUTPUT:

```

A4 : {'OR': ['A10']} -> {'A10': 1}
A3 : {'OR': ['A7'], 'AND': ['A8', 'A9']} -> {'A8 AND
A9': 2, 'A7': 5}
A2 : {'OR': ['A5', 'A6']} -> {'A5 OR A6': 9}
A1 : {'OR': ['A2'], 'AND': ['A3', 'A4']} -> {'A3 AND
A4': 5, 'A2': 10}
Shortest Path:
A1<<-(A3 AND A4) [A3<<-(A8 AND A9) [A8 + A9] + A4<<-
A10]

```

Here, weight of all edges is considered to be 1. For the given graph, first path from A1 -> A2 and A1->(A3+A4) is explored which is 7 and 10 respectively. So, A1 -> A2 is followed. Then, A2-> A5 and A2-> A6 is explored and weight of A2 is updated be min (9, 10). Then possible paths from a A3 are explored and its weight is updated as 2. Similarly, weight of A4 is updated to 1 after path exploration. Finally weight of A1 is updated considering updated weights of A2, A3 and A4.

11.4.8 Iterative Deepening A*

The IDA* algorithm (explained in 3.10.1) combines the optimality of A* with the space efficiency of DFS. It begins with a cost limit equal to the heuristic of the start node. Then it performs a depth-first search but only expands nodes where the total cost $f(n) = g(n) + h(n)$ is less than or equal to the current limit. If the goal is not reached, the limit is increased to the smallest $f(n)$ that exceeded the previous bound and the search is restarted. In the graph given in figure 3, the algorithm starts from "A" with a bound of 4. It first tries exploring "B" and "C", but the path "A → C → F" is found within the bound and is chosen as the optimal path. This method avoids storing all frontier nodes like A*, thus saving memory while still guaranteeing an optimal solution. IDA* is widely used in puzzle-solving (like 8-puzzle, 15-puzzle) and in AI search problems where memory is critical.

```

#CODE: Iterative Deepening A*

graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 2), ('E', 5)],

```

```

    'C': [('F', 1)],
    'D': [],
    'E': [('F', 1)],
    'F': []
}

heuristics = {
    'A': 4,
    'B': 2,
    'C': 1,
    'D': 5,
    'E': 1,
    'F': 0
}

def ida_star(start, goal):
    def dfs(node, g, bound, path):
        f = g + heuristics[node]
        if f > bound:
            return f, None
        if node == goal:
            return True, path
        min_bound = float('inf')
        for neighbor, cost in graph[node]:
            if neighbor not in path: # avoid cycles
                result, new_path = dfs(neighbor, g +
cost, bound, path + [neighbor])
                if result is True:
                    return True, new_path
                elif result < min_bound:
                    min_bound = result
        return min_bound, None

    bound = heuristics[start]
    path = [start]
    while True:
        result, found_path = dfs(start, 0, bound,
path)
        if result is True:
            return found_path
        if result == float('inf'):
            return None
        bound = result

# Run IDA*
start, goal = 'A', 'F'
solution = ida_star(start, goal)
print(f"IDA* Path from {start} to {goal}:",
solution)

```

OUTPUT:

IDA* Path from A to F: ['A', 'C', 'F']

Check your Progress 3

1. For the 8-puzzle problem, two heuristics are given: h1: Number of misplaced tiles and h2: Sum of Manhattan distances. Which heuristic will expand fewer nodes and why?

.....
.....
.....

2. In DFS, what happens when the algorithm reaches a node with no unvisited neighbours?

.....
.....
.....

3. An AI agent must find the shortest route in a city map where nodes represent intersections and edges represent roads with distances. Design this as a state space search problem. Explain why A* is preferred over BFS.

.....
.....
.....

11.5 MODEL EVALUATION AND HYPERPARAMETER TUNING IN PYTHON

Once we build a machine learning model, we must evaluate its performance to check if it is accurate and reliable. Common Evaluation Metrics for classification problems are: accuracy, precision, recall, F1-Score, confusion matrix and ROC-AUC Score.

A confusion matrix is a 2×2 (for binary classification) table that summarizes model predictions against actual outcomes. It is represented as follows:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

It helps to visualize where the model is making errors. Using confusion matrix, we can compute accuracy, precision, recall, etc.

Accuracy measures the overall correctness of a model. It tells how many total predictions were correct out of all predictions made.

$$\text{Accuracy} = \{\text{TP} + \text{TN}\} / \{\text{TP} + \text{TN} + \text{FP} + \text{FN}\}$$

Precision measures how many of the predicted positive cases were actually positive. It answers: *“When the model says ‘yes’, how often is it correct?”*

$$\text{Precision} = \{\text{TP}\} / \{\text{TP} + \text{FP}\}$$

Recall measures how many of the actual positive cases were correctly predicted. It answers: *“Out of all the real positives, how many did we find?”*

$$\text{Recall} = \{\text{TP}\} / \{\text{TP} + \text{FN}\}$$

The F1-Score is the harmonic mean of precision and recall. It balances both metrics — useful when you need to find a trade-off between precision and recall.

$$\text{F1-Score} = (2 * \{\text{Precision}\} * \{\text{Recall}\}) / (\{\text{Precision}\} + \{\text{Recall}\})$$

ROC (Receiver Operating Characteristic) curve plots True Positive Rate (Recall) vs False Positive Rate (FPR) at different threshold values. AUC (Area Under the Curve) measures how well the model distinguishes between classes. If AUC = 1.0, it means classifier is a perfect one. And if AUC = 0.5, it means, there is no discrimination (random guessing).

Sklearn provides the function support to find these metrics.

Hyperparameter Tuning - Machine learning models have hyperparameters (settings chosen before training, not learned from data). Examples:

- Number of trees in RandomForest.
- Learning rate in XGBoost/Gradient Boosting.
- Kernel type in SVM.

Choosing the right hyperparameters can dramatically improve performance.

Methods of Hyperparameter Tuning

- Grid Search – Tries all combinations of parameters (refer to 11.3.4 for implementation and syntax).
- Random Search – Tries random combinations (faster).
- Bayesian Optimization / Optuna – Smarter search (advanced).

11.6 SUMMARY

In this unit, we learned about implementation details along with different libraries of Python which are useful for writing AI and ML based code. Further, we have seen Python code for different AI based algorithms such as N-Queen problem, Min-Max algorithm with and without alpha beta pruning, BFS, DFS, Best First search, A*, AO* and IDA* search algorithms. Also, the unit covered the discussion about the evaluation metrics used for classification problems. Finally, the unit discussed the methods of hyperparameter tuning available in python.

11.7 SOLUTIONS/ANSWERS

Check your Progress 1

1.

```
import matplotlib.pyplot as plt

# Sample data
labels = ['AI', 'ML', 'Data Science', 'Cloud', 'IoT']
sizes = [30, 25, 20, 15, 10]

# Draw pie chart
plt.figure()
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title('Sample Technology Distribution')
plt.show()
```

2.

```
arr = np.array([30, 25, 20, 15, 10])
print(np.sum(arr))
print(np.mean(arr))
print(np.std(arr))
print(np.max(arr))
print(np.min(arr))
```

3. Tensors are multidimensional arrays that can store numerical data — similar to NumPy arrays. They help in deep learning because:

- I. They efficiently represent input data in numerical form.
- II. They support automatic differentiation, which helps in computing gradients during backpropagation.
- III. They can be easily processed on GPUs, enabling faster training of deep learning models.

Check your Progress 2

1. The `is_safe()` function checks if a queen can be safely placed in a specific row and column. It ensures no other queen is already placed in the same column, row, or diagonals, to prevent attacks between queens.
2. When a position is not safe, the algorithm skips that position and tries the next column (or next row). This process continues until a safe position is found or all possibilities are exhausted, leading to backtracking.
3. The base case occurs when the game reaches a terminal state, that is, when the game is over (win, lose, or draw) or when there are no more possible moves. At this point, the algorithm returns the evaluated score of that state instead of continuing recursion.

Check your Progress 3

1. When A^* is applied using given heuristic functions, Manhattan distance heuristic (h_2) will expand fewer nodes because it provides a more accurate estimate of the remaining cost and dominates the misplaced-tile heuristic (h_1).
2. The algorithm backtracks to the previous node to explore other unexplored paths.
3.
State space: Each state represents the current intersection where the AI agent is located.
Initial state: The starting intersection S.
Goal state: The destination intersection G.
Actions / Operators: From a given intersection, the agent can move to any adjacent intersection connected by a road.
Transition model: Applying an action moves the agent from one intersection to another along a road.
Path cost function: The cost of a path is the sum of distances of the roads traversed.
Objective: Find a path from SSS to GGG with minimum total distance.

A^* is preferred over BFS for city route planning because it accounts for varying road distances, guarantees an optimal solution using heuristics, and explores the search space more efficiently.

11.8 FURTHER READINGS

1. S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2021.
2. Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
3. Burkov, The Hundred-Page Machine Learning Book, Quebec, Canada: Andriy Burkov, 2019.